

Jimmy Lei Ba (Apple) Image Resnet 101 Spill of the beans

Training GPT DNNs is expensive

one way to reduce the training time is to normalize the activities of neurons

Batch normalization: over distribution of the summed inputs to a neuron over a mini batch of training cases to compute a mean and a variance

Significantly reduces time in training FFNNs, but effect of batch norm depends on mini batch size $\rightarrow$ not obvious how to apply it to recurrent neural networks which are also used to normalize the summed input to that neuron on each training case

How compute mean & variance used for normalization from all the summed inputs to the neuron on a layer on a single training case

Like batch norm $\rightarrow$ give each neuron its own adaptive bias and gain which are applied after the normalization but before the non-linearity

Unlike batch norm

$\rightarrow$ layer norm performs exactly the same computation at training & test times

Layer norm $\rightarrow$ straight forward to apply to recurrent neural networks by computing the normalization stats at each step

$\rightarrow$ Very effective at stabilizing the hidden state dynamics in recurrent networks

Layer norm computes statistics across neurons within one example
 $\times$ not across batch

LN normalizes across $\rightarrow$ feature dimension

eg Sample $x = [x_1, x_2, \dots, x_n]$

Mean & variance computed over $x_1, x_2, \dots, x_n$

BN $\rightarrow$ stabilize a neuron's re-distributions across dataset

LN $\rightarrow$ stabilize representation vector scale per sample

intuition (normalize the representation vector length)

Why LN in transformers?

Seq length vary
batch sizes vary
 $\rightarrow$ train processed independently

$\times$ Batch norm would mix statistics across tokens in batch!

Batch Norm computes distribution per neuron

eg for neuron 1 across batch samples

sample	neuron's value
1	2.1
2	2.7
3	2.8

mean/variance computed only for that neuron

Every neuron has its own mean & variance

* BN mainly helps w/ Internal covariate shift

Distribution of activations keep changing as values layers update

$\rightarrow$ smoothing opt landscape
allowing bigger learning rates
improving gradient conditioning

Gradient flow

Batch Norm gradients depend on other samples in the batch

Layer Norm " " only on the sample itself

Batch Norm $\rightarrow$ baseline technique before Layer Norm

Summed input to a neuron means

if $z_i = w_i^T x + b_i \rightarrow$ opt vector

(production / summed input) $\rightarrow$ bias

egs of our batches $\rightarrow \{x^{(1)}, x^{(2)}, \dots, x^{(n)}\}$

each neuron produces $z_1^{(1)}, z_1^{(2)}, \dots, z_1^{(n)}$

Across the batch, the neuron produces a distribution of values

it has mean $\mu_i = \frac{1}{B} \sum_{r=1}^B z_i^{(r)}$

Variance $\sigma_i^2 = \frac{1}{B} \sum_{r=1}^B (z_i^{(r)} - \mu_i)^2$

Each value is normalized

$\hat{z}_i^{(r)} = \frac{z_i^{(r)} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$ (the distribution becomes mean = 0, variance = 1)

Real limitation of BN (depends on batch size)
Information leakage across examples
(output for one sample depends on another sample)

In RNNs $\rightarrow$ Seq length varies
batch sizes may be small
(in steps depend on previous state)

Batch stats unstable with small batch
Each step may need separate statistics

Need to apply when sequence processed step by step

Batch norm instead of forcing about standardization adds learnable params

$y_i^{(n)} = \gamma_i x_i^{(n)} + \beta_i$
 $\rightarrow$ shift
 $\rightarrow$ scale

$\rightarrow$ so the network can learn optimal scale

Before BN $x \rightarrow$ linear $\rightarrow$ activation

with BN $x \rightarrow$ linear $\rightarrow$ ϵ

$\downarrow$ normalization
 $\downarrow$ scale shift
 $\downarrow$ activation

Why BN can't?

problem Internal covariate shift

Distribution of neuron input keeps changing because values layers update
 $\rightarrow$ optimization becomes unstable
training becomes slow

BN stabilizes the distribution

Gradient flow

Normalization controls magnitude of activations (prevents exploding activations)
very strong gradients
unstable optimization

If BN is used

effects $\rightarrow$ deeper

losses training
higher sensitivity to initialization
unstable gradients

BN normalizes across examples, not within a single example
Dimension $\rightarrow$ batch dimension

Layer Norm changes this

BN normalizes per neuron across batch

same neuron
different samples

LN normalizes

same sample
across neurons

split with activation before or after forward

BUT: issue of normalization

for a layer with

Batch size = B
Neurons / features = H

The preactivation matrix looks like $Z \in \mathbb{R}^{B \times H}$

	neuron 1	neuron 2	neuron 3
sample 1	z_{11}	z_{12}	z_{13}
sample 2	z_{21}	z_{22}	z_{23}
sample 3	z_{31}	z_{32}	z_{33}
sample 4	z_{41}	z_{42}	z_{43}

Batch norm statistics for each neuron (columns from that example matrix)

Layer norm flips the axis $\mu_1 = \frac{1}{B} \sum_{i=1}^B z_{i1}$ $\sigma_1^2 = \frac{1}{B} \sum_{i=1}^B (z_{i1} - \mu_1)^2$ } neurons } norm all samples } Normalization happens column wise

for each sample (row) $\mu_1 = \frac{1}{H} \sum_{j=1}^H z_{1j}$ $\sigma_1^2 = \frac{1}{H} \sum_{j=1}^H (z_{1j} - \mu_1)^2$ } sample, across all neurons } Normalization happens row wise

Batch norm
→ normalize vertical direction
→ good for CNN's
→ large batches
→ stabilizes distinct statistics per neuron

Layer norm
→ Normalize horizontal direction
→ stable layer representation within of each sample
→ good for RNN's
→ variable batch sizes

Transformers could technically use batch norm
but problem arise because
→ sequence lengths vary
→ batch sizes vary
→ tokens processed independently

BUT → training time equivalent

→ the stochasticity from batch statistics serves as a regularizer during training

mean & variance are calculated from a row's batch, not the whole dataset

for neuron 1: $\mu_1 = \mu_{\text{row}}(z_{1:}^T)$

Not batch change every iteration
 μ_1, σ_1^2 changes even if input is same

this injected noise into the network during training
→ prevents model from overfitting hence acts as regularizer

Information flow dependent

BUT stochastic regularization is forward pass

→ input → linear → batch stats → normalized activations

since statistics vary

→ forward pass slightly changes each iteration

if we compute mean/variance over entire dataset every step
no noise

Training would still be stable

Regularization disappears

Batch Norm helps training for

(1) optimization improvement

better gradient conditioning

larger learning rates

faster convergence

(2) Implicit regularization

noise from batch stats

reduces overfitting

This stochasticity becomes a problem for

small batch sizes, RNNs, sequence models

because batch statistics become very noisy

noisy statistics

Batch size $\Rightarrow$ more numerous statistics

normalized activation of a sample $\propto$ 1/n batchnorm depends on all other samples (coupling effect)

why BUT makes training converge faster?

→ scale of activations normally change during use diff become

→ unless layers change
→ weights change
→ distributions shifts } variable gradient magnitudes
} poor conditioning of the loss landscape

Batch Norm
→ so that activations normally stay in the same range

mean=0
variance=1
Optimization convergence
(The loss surface becomes smoother and better conditioned)
→ we can use large learning rates
→ which would never faster convergence with SGD

Gradient bias (Normalization removes arbitrary scaling of weights)

normally if σ doubles → activation distribution changes

with BN → normalization smooth scale change
(model becomes less sensitive to parameter scale)

if no neuron layer norm from transformers → training instability
gradient explosion
extremely sensitive learning rates

pipeline → x
↓
linear (w, b)
↓
normalization (using μ, σ)
↓
scale + shift (x, μ, σ)
↓
activation

why? if only pure normalization
every neuron output will always have mean=0
variance=1 for every layer

solution: learnable scaling

after normalization

$z = \gamma \tilde{z} + \beta$
learnable shift
variable scale

if the neuron isn't really $\mu=0, \sigma=1$ because that scale works well for next layer

pure Normalization would force it to 0 & 1 only

Model loses expressive flexibility

now the network can learn
mean = μ
variance = σ^2

so normalization becomes

standardize → then allow the network to choose scale again

layer norm removes this global properties

Global shift

if every element increases by same constant

→ layer norm subtracts the mean, so this shift disappears

if vector is multiplied by a constant

→ division by standard deviation removes scaling

2 Background

FFN $\xrightarrow{\text{non-linearity}}$

L^2 hidden layer is a deep feed forward NN

a^l → vector representations of the summed inputs to the neurons in that layer

summed inputs are calculated/computed through a

linear projection with weight matrix W^l
and bias term b^l

$a^l = W^l \cdot a^{l-1}$
 $b^l = f(a^l + b^l)$
non-linearity
→ scales bias parameter

they define batchnorm as $a_i^l = \frac{1}{\sqrt{B}} \sum_{j=1}^B (a_j^l - \mu^l)$ $\mu^l = \frac{1}{B} \sum_{j=1}^B a_j^l$
 $\sigma^{l^2} = \frac{1}{B} \sum_{j=1}^B (a_j^l - \mu^l)^2$
→ normalized of summed inputs to the l th hidden unit in the l th layer

3 Layer Normalization

$a_i^l = \frac{1}{\sqrt{H}} \sum_{j=1}^H a_j^l$ $\sigma^l = \sqrt{\frac{1}{H} \sum_{j=1}^H (a_j^l - \mu^l)^2}$

→ no of hidden units in a layer

All hidden units in a layer share the same normalizing mean μ and σ
but diff training cases have different normalization mean

3.1 Layer Normalization of RNNs

→ standard of RNNs

summed inputs in the recurrent layer

→ layer norm computes statistics within the hidden vector at that step, not across time or batch
at time t

3.1 Layer normalization of RNNs

in standard RNNs

summed inputs in the recurrent layer

↳ computed from current input x^t and previous vector of hidden states h^{t-1}

$$w^T = W_{hh} h^{t-1} + W_{xh} x^t$$

The layer normalization of recurrent layer no-center & no-scale its activations using the entire normalization term

$$h^t = f \left[\frac{1}{\sigma} \left(0 \cdot (h^t, u^t) + b \right) \right] \quad u^t = \frac{1}{H} \sum_{i=1}^H a_i \quad \sigma = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^2 - u^2)}$$

W_{hh} → recurrent hidden to hidden weights

W_{xh} → bottom up input to hidden weights

> layer norm computes statistics within the hidden vector at that step, not across time or batch

$$\text{At time } t \quad a^t = [a_1^t, a_2^t, \dots, a_H^t]$$

$$\text{then compute } u^t = \frac{1}{H} \sum_{i=1}^H a_i^t \quad \sigma^t = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^t - u^t)^2}$$

Normalization is local to the current time step

No dependence on
batch
sequence length
than time steps

4 Related work

Normalization terms depend only on the summed inputs at the current time-step

practically Batch Norm to RNNs

↳ key finding → BN works better as RNNs of each time step has its own statistics

↳ problem → must maintain separate normalization statistics per time step (becomes awkward when sequence lengths vary)

Initializing gain param to 1 significantly improved performance in recurrent batch norm setup (just in training phase)

One idea is weight normalization

Both BN & LN effectively reparametrize the network

meaning they change how parameters are represented

without changing the underlying function class

But authors emphasize LN is not just another reparametrization

↳ it changes the model that introduces different invariance properties

(Layer norm changes how activations behave across a layer)

5 Analysis

5.1 invariance under weights and data transformations

→ normalizing summed inputs a_i to a mean through the two scales μ and σ

The methods also have adaptive bias & gain g for each neuron after the normalization

$$h_i = f \left(\frac{g \left(\frac{a_i - \mu}{\sigma} \right) + b_i}{\sigma} \right)$$

Note in LN normalization $\mu=0$ $\sigma=1/\sqrt{2}$

	LN	WN	LN	WN	LN	WN
LN	Yes	No	Yes	No	Yes	No
WN	No	Yes	No	Yes	No	Yes

Inv. invariance

wt re scaling and recentering

model param b, b' where u^t matrices $W \neq W'$

Scaling factor $\delta \Rightarrow u' = \delta u + \gamma \cdot \mathbf{1}$

W is shifted by γ

$$h' = f \left(\frac{g \left(\frac{W'x - u'}{\sigma'} \right) + b'}{\sigma'} \right) = f \left(\frac{g \left(\frac{\delta Wx + \gamma \cdot \mathbf{1} - u}{\delta \sigma} \right) + b'}{\delta \sigma} \right) = f \left(\frac{g \left(\frac{Wx - u}{\sigma} \right) + b}{\sigma} \right) = h$$

Data rescaling & recentering

x' new data point obtained by rescaling x by δ

$$h'_i = f \left(\frac{g \left(\frac{W'x' - u'}{\sigma'} \right) + b'}{\sigma'} \right) = f \left(\frac{g \left(\frac{\delta Wx - \delta u + \gamma \cdot \mathbf{1} - u}{\delta \sigma} \right) + b'}{\delta \sigma} \right) = h_i$$

Weight normalization

Instead of normalizing activations, it normalizes weights

standard reason $a = Wx$

WN rescales weight vectors

$w = \frac{g}{\sqrt{H}}$ so its L2 norm of the wt vector is controlled

wt rescaling → u is now explicitly normalized w.r. to its magnitude
batch norm

$$a_i = W_i^T x$$

$$a'_i = \delta a_i \quad \text{BN computes } \mu = \delta u$$

$$\sigma = \delta \sigma$$

Normalized value doesn't change

layer norm → if individual neurons are scaled, then the sample representation is normalized vector change
X not invariant to individual/neuron scaling

5.2 Geometry of parameter space during learning

→ normalization scalar can implicitly reduce the learning rate and makes learning more stable

5.2.1 Riemann metric

The Riemann metric is a statistical model from a smooth that comes of all the possible input-output realizations of the model. For models whose output is a probability distribution, a natural way to measure separation of two points would be KL divergence, but then model output distributions

Under KL divergence metric, the parameter space is a Riemann manifold

curvature of this is entirely captured by its Riemann metric, where gradients from a shared set

act as the infinitesimal distance in the tangent space at a point in the parameter space

They ask if two models represent the same function do they learn differently? YES!!!

→ Normalization changes the geometry of parameter space, which changes optimization behavior

They just want to show that Bayesian & Batch norm modify the effective learning rate and stabilize training

2 parameter space intuition

Imagine a model with 2 parameters

$$\Theta = (w_1, w_2)$$

↳ all possible pairs of parameters produces a different model

So the parameter form a space of models

the measure separation of two points would be KL divergence, but their model output distributions

Under KL divergence metric, the parameter space is a Riemann manifold

structure of space is entirely captured by its Riemann metric, where quadratic form is denoted as ds^2

ds^2 is the infinitesimal distance in the tangent space at a point in the parameter space

Intuitively, it measures the changes in the model output from the parameter space along a tangent direction

The Riemann metric under KL was previously studied and shown to be well approximated under second order Taylor expansion using the Fisher information matrix

$$ds^2 = D_{KL}[P(y|x, \theta) \| P(y|x, \theta + d\theta)] \approx \frac{1}{2} d\theta^T F(\theta) d\theta$$

θ small change in parameters

$$F(\theta) = E_{x \sim P(x), y \sim P(y|x)} \left[\frac{\partial \log P(y|x, \theta)}{\partial \theta} \frac{\partial \log P(y|x, \theta)}{\partial \theta}^T \right]$$

5.1.2 Geometry of normalized generalized linear models

following analysis can be applied to understand deep neural networks with block diagonal approximation to the Fisher information matrix, where each block corresponds to the parameter for single neuron

GLM can be regarded as parametrizing an output distribution from the exponential family using a weight vector w and bias scalar b

The log likelihood of the GLM can be written using the summed inputs x as the following

$$\log P(y|x, w, b) = \frac{(w^T x + b - \eta(w, b))}{\phi} + \log \phi$$

$$E(y|x) = \eta(w, b) = \phi(w^T x + b), \text{Var}(y|x) = \phi(w^T x + b)$$

Assume H dimensional output vector

$y = [y_1, \dots, y_H]^T$ modeled using H independent GLMs

$$\text{and } \log P(y|x, w, b) = \sum_{i=1}^H \log P(y_i|x, w_i, b_i)$$

The Fisher information matrix for the multi-dimensional GLM w.r.t its parameters

$$\Theta = [w_1^T, w_2^T, \dots, w_H^T, b_1, b_2, \dots, b_H]^T = \text{Vec}([w_i, b_i])$$

Simply the repeated Kronecker product of the data features & the output covariance matrix

$$F(\theta) = E_{x \sim P(x)} \left[\frac{w^T(x)}{\phi^2} \otimes \begin{bmatrix} x x^T & x \\ x^T & 1 \end{bmatrix} \right]$$

We then normalize GLMs by applying the normalization methods to the summed inputs x in the original model through x_1 & σ

Without loss of generality, we denote F as the Fisher information matrix under the normalized multi-dimensional GLM with the additional joint parameters $\Theta = \text{Vec}([w, b, \sigma])^T$

$$\Theta = \text{Vec}([w, b, \sigma])^T$$

$$F(\Theta) = \begin{bmatrix} F_{11} & F_{1H} \\ F_{H1} & F_{HH} \end{bmatrix}, F_{ij} = E_{x \sim P(x)} \left[\frac{\text{Cov}([y_i, y_j | x])}{\phi^2} \begin{bmatrix} \frac{\partial \log P(y_i | x)}{\partial \sigma} & \frac{\partial \log P(y_i | x)}{\partial w_i} & \frac{\partial \log P(y_i | x)}{\partial b_i} \\ \frac{\partial \log P(y_j | x)}{\partial \sigma} & 1 & \frac{\partial \log P(y_j | x)}{\partial w_j} \\ \frac{\partial \log P(y_j | x)}{\partial w_j} & \frac{\partial \log P(y_j | x)}{\partial w_j} & \frac{\partial \log P(y_j | x)}{\partial b_j} \end{bmatrix} \right]$$

$$X_1 = x - \frac{\partial w_1}{\partial \sigma} = \frac{w_1 - w_1}{\sigma_1} \frac{\partial \sigma_1}{\partial w_1}$$

Improved learning rate reduction through growth of weight vector

→ compared to standard GLM, F_{11} block along the w_1 vector is scaled by σ_1 given parameter normalization scalar σ_1

→ if the norm of weight vector w_1 grows twice as large even though model output remains the same, Fisher information matrix will be different

→ sensitive along w_1 will change by a factor of $1/2$ because the σ_1 will also be twice as large

→ As a result, for the same parameter update in the normalized model, the norm of w_1 vector effectively controls learning rate for w_1 vector

→ During learning, it's hard to change w_1 vector initialization with large norm. The normalization methods therefore have an implicit "early stopping" effect on the w_1 vector and help to stabilize learning towards convergence

Learning the magnitude of growing weights in normalized models it is explicitly parametrized by joint parameters

We compare how the model output changes between updating joint parameters in the normalized GLM

updating the magnitude of the equivalent weights under original parameterization during learning

Imagine a model with n parameters

$$\Theta = (w_1, w_2)$$

→ all possible pairs of parameters produces a different model

So the parameters form a space of models

Parameter Space → Every point = one model

Training = moving in this space

3 Why distance matters?

Suppose we slightly change parameters

$$\Theta = \theta + d\theta$$

how much does model output change?

Two parameter changes could be the same size but produce very different output effects

e.g. small w_1 change = huge prediction change

small w_2 change = something prediction change

So we need a meaningful way to measure meaningful distance in parameter space

4 KL divergence also says so

The dist. between two models = difference between their output distributions

measuring KL divergence

$$D_{KL}(P(y|x, \theta) \| P(y|x, \theta + d\theta)) \rightarrow \text{how different are the predictions of two models}$$

5 Informational change

Assume parameter change is small

Then KL divergence can be approximated by a quadratic form

$$ds^2 \approx \frac{1}{2} d\theta^T F(\theta) d\theta$$

Interpretation

→ distance = quadratic function of parameter change

Matrix $F(\theta)$ determines the curvature of parameter space

6 Fisher Information Matrix

$$F(\theta) = E \left[\left(\frac{\partial \log P(y|\theta)}{\partial \theta} \right) \left(\frac{\partial \log P(y|\theta)}{\partial \theta} \right)^T \right]$$

Fisher matrix measures sensitivity of the model output to the parameter changes

High values → model output changes a lot when parameters change

Low values → Output barely changes

Fisher matrix just tells us the geometry of the parameter space

7 Riemann Geometry

Riemann Manifold

→ parameter space where distance is defined by the Fisher matrix

Instead of euclidean distance

distance between models = KL divergence approximation

8 why they analyze a GLM

(because it is mathematically tractable)

GLM form is $w^T x + b$ like a neuron

→ is basically a single neural unit with probabilistic output

9 by likelihood equation

$$\log P(y|x, w, b)$$

→ log-likelihood is exponential family form

$$(w^T x + b) \eta - \eta(w^T x + b) \rightarrow \text{standard GLM form}$$

Key idea: Model output depends on summed input x

10 Fisher matrix for this model

$$F(\theta) = E \left[\text{Cov}(\eta(x)) / \phi^2 \right] * [x x^T, x, 1]$$

→ what matters conceptually → Fisher matrix depends on the input vector x

(consequence of neural network same as neural network units & weights)

version is more capturing variation because parameters

Expectation $E(\cdot)$ Average over the data distribution

So Fisher matrix is basically

average sensitivity of model/likelihood to parameters

Why it appears a geometry?

in euclidean space distance is $ds^2 = d\theta^T d\theta$

in model space the meaningful distance is $ds^2 = d\theta^T F(\theta) d\theta$

Thus the version matrix they mention

measuring distance between models depends on Fisher matrix

Why this matters for optimization?

Gradient descent assumes parameter space is flat (equal step size everywhere)

Fisher Matrix says some directions change the model a lot
some directions barely change it

So the geometry is curved

and minimization methods modify the curve

What this paper is doing?

↳ they analyse how normalization affects Fisher matrix

↳ reveals the curvature of parameter space which effectively changes learning rate in different directions

Whereas we see

$$F(\theta) = E[\text{output} \cdot \text{output}^T] \text{ or } ds^2 = d\theta^T F d\theta$$

it means they are discussing

information geometry / natural gradient / optimization curvature

Whereas Fisher matrix appears + translate it to

Matrix describing how model predictions to parameter change

$$\text{they define } \chi_i = x - \frac{\partial u_i}{\partial w_i} = \frac{a_i - u_i}{\sigma_i} \frac{\partial u_i}{\partial w_i}$$

↳ gradient of normalized activation

Because u and σ depend on the weights
So when computing derivatives, we must include how u and σ change
This is the standard chain rule

* Implicit leaves side reduction through the growth of the weight vector

↳ if weight magnitude increases $|w| \uparrow$ then σ increases

Which means the number of activations divided by a larger number

↳ so effective update step becomes smaller

Training stabilization

without norm $\rightarrow$ larger wts $\rightarrow$ large gradients $\rightarrow$ unstable updates

with normalization $\rightarrow$ large wts $\rightarrow$ larger $\sigma \rightarrow$ gradients / down

↳ the model automatically slows down when weights grow

through s.s. they prove

adaptive scaling of parameter updates

↳ which stabilizes training

Remember the chain

KL divergence $\rightarrow$ Fisher matrix $\rightarrow$ geometry of parameter space

Normalization Modifies the Fisher Matrix

↳ changes optimization behaviour